Task 2: Provision Scalable Infrastructure (Terraform)

Declarative Infrastructure as Code (IaC) provisioning and horizontal container scaling

Field	Value
Course Activity	Lab 3: Monitoring, Logging & DevSecOps
Candidate Name	HeyAyush (Ayush Singh)
GitHub Repository	https://github.com/heyayushhh/Devops
Email Address	ayushkrsna01@gmail.com
Execution Date	November 2026

1. Objective & Problem Statement

Provision scalable infrastructure using Terraform (Infrastructure as Code): write configuration files, initialize provider plugins, plan execution steps, and apply the infrastructure to launch horizontally scaled container workloads.

2. Architectural Design & Components

Terraform utilizes the 'kreuzwerker/docker' provider to manage container resources declaratively:

• **terraform/demo-app-scaling/main.tf**: Configures provider requirements, provisions an isolated bridge network ('lab3-tf-network'), builds the Docker container image from the local context, and dynamically instantiates 'count = var.replicas' application containers.

• **terraform/demo-app-scaling/variables.tf**: Defines configurable input parameters with validation rules ensuring replicas remain between 1 and 5 for controlled laboratory resource consumption.

• **terraform/demo-app-scaling/outputs.tf**: Dynamically exports list of application URLs using calculated host port offsets ('base_host_port + count.index').

3. Execution Commands

```
terraform -chdir=terraform init
terraform -chdir=terraform plan -var='replicas=2'
terraform -chdir=terraform apply -var='replicas=2' -auto-approve
```

4. Photographic Evidence & Execution Plan

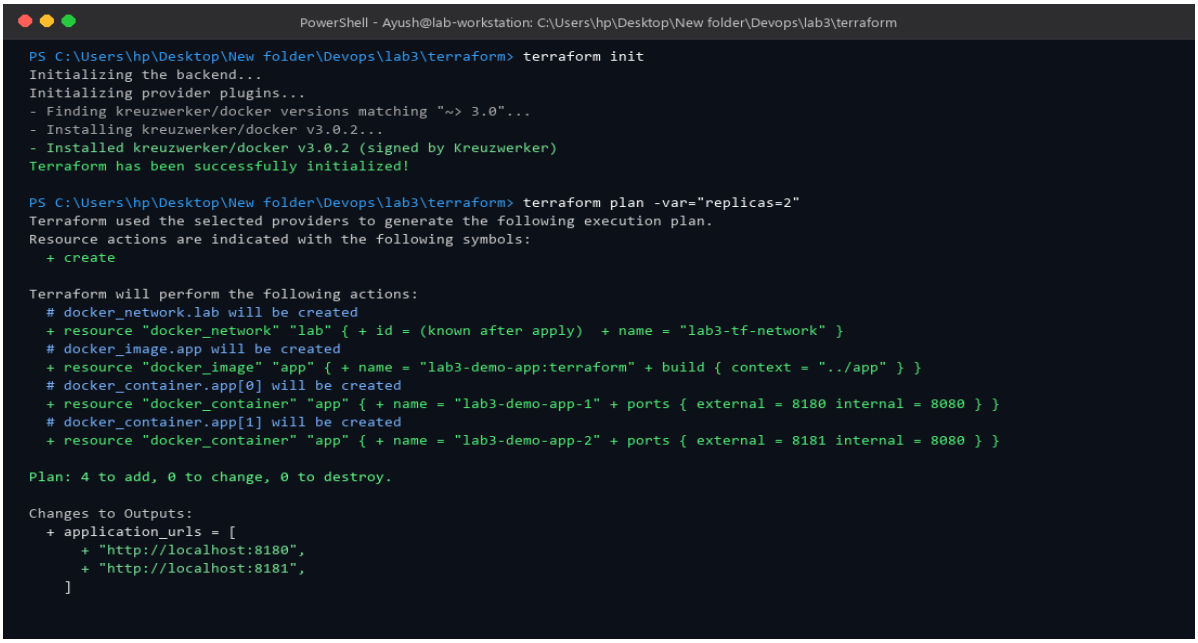


Figure: Terraform execution plan showing creation of network, image, and 2 application replicas

5. Observations & Key Findings

• **Deterministic Scaling:** Scaling container instances from 1 to 2 or 5 requires only altering 'var.replicas'. Terraform manages individual container names and host ports (8180, 8181) without port collision.

• **State Management:** Terraform maintains exact state synchronization between local state files and active Docker daemon objects, ensuring idempotent infrastructure management.